

Ball Machine

Madina đã phát minh ra một máy bắn bóng để giải trí cho những thí sinh của IOI. Cấu trúc bên trong của máy như sau:

- Máy này bao gồm N **nút**, được đánh số từ 0 đến $N - 1$. Nút $N - 1$ được gọi là **nút gốc**.
- Với mỗi nút u mà $0 \leq u < N - 1$, **nút cha** của nút u là một nút $P[u]$ có $P[u] > u$, và nút u là **nút con** của $P[u]$. Nút gốc không có nút cha. Một nút không có nút con được gọi là **nút lá**.

Bạn **không** biết N và không biết nút cha $P[u]$ của bất kỳ nút u nào. Thay vào đó, Madina cho bạn biết M là số lượng nút lá, và chỉ số của các nút lá là $0, 1, \dots, M - 1$. Nhiệm vụ của bạn là xác định cấu trúc bên trong của máy bằng cách vận hành nó.

Mỗi nút của máy có thể chứa tối đa một **quả bóng**, và mỗi quả bóng có một **giá trị** là một số nguyên không âm. Ta nói rằng một nút có giá trị x nếu nó chứa một quả bóng có giá trị x . Một nút được coi là **bị chiếm dụng** nếu nó chứa một quả bóng; ngược lại, nó là **trống**. Ban đầu, mọi nút đều trống.

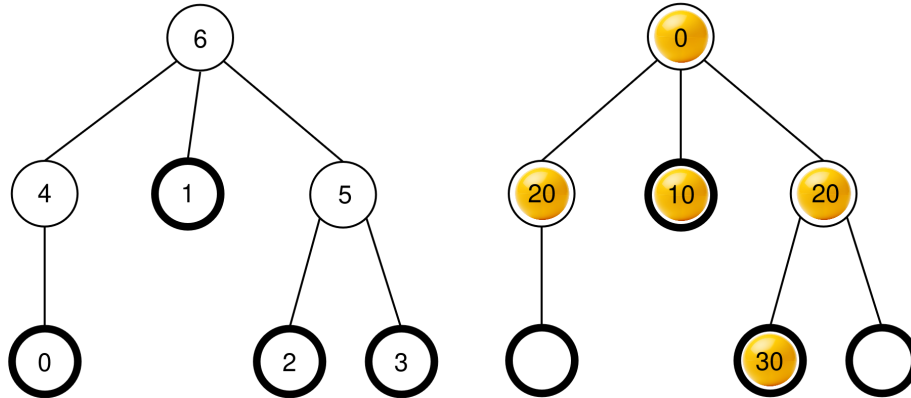
Bạn có thể thực hiện hai loại thao tác:

- insert** (U, X) : Chèn một quả bóng mới có giá trị là X vào nút lá U .
 - Nếu nút lá U đã bị chiếm dụng, thao tác sẽ trả về `false` và không chèn quả bóng.
 - Nếu nút lá U trống, quả bóng được đặt tại nút U . Sau đó, quả bóng liên tục di chuyển đến nút cha của nút hiện tại miễn là nút cha đó trống. Quá trình di chuyển dừng lại khi quả bóng đến nút gốc hoặc một nút có nút cha đang bị chiếm dụng. Thao tác trả về `true`.
- collect()**: nếu nút gốc trống (nghĩa là máy trống), hàm `collect` trả về một mảng rỗng. Ngược lại, hàm đệ quy `traverse` được mô tả bên dưới sẽ thu thập giá trị của tất cả các quả bóng hiện có trong máy vào một mảng S . Ban đầu, mảng S trống và `traverse(N - 1)` được gọi.

```
traverse(u) :  
    thêm giá trị của quả bóng tại nút u vào cuối S  
    gọi c[u] là danh sách các nút con bị chiếm dụng của nút u.  
    sắp xếp c[u] theo thứ tự không giảm dựa trên giá trị  
    của các quả bóng tại các nút đó (nếu có nhiều nút giá trị giống  
    nhau, chúng có thể xuất hiện theo bất kỳ thứ tự nào)  
    for mỗi v thuộc c[u]:  
        traverse(v)
```

Sau khi hàm này kết thúc, **tất cả các quả bóng sẽ được lấy ra** khỏi máy và `collect` sẽ trả về S .

Ví dụ, xét một máy có $N = 7$ nút và mảng cha $P = [4, 6, 5, 5, 6, 6]$. Hình bên trái hiển thị máy trông với các chỉ số nút, trong khi hình bên phải cho thấy một cấu hình khả thi của các quả bóng sau một dãy các thao tác `insert` (dãy này được đưa ra trong phần Ví dụ).



Khi `collect()` được gọi, nút gốc (nút 6) bị chiếm dụng, do đó S được khởi tạo là $S = []$ và `traverse(6)` được gọi.

`traverse(6)` : nút 6 có giá trị 0; thêm nó vào S sẽ cho $S = [0]$. Các nút con của nút 6 là 4, 1 và 5, tất cả đều đã được chiếm dụng. Giá trị của chúng lần lượt là 20, 10 và 20. Sắp xếp theo thứ tự không giảm sẽ cho $c[6] = [1, 5, 4]$ (lưu ý rằng $c[6] = [1, 4, 5]$ cũng hợp lệ, vì các nút 4 và 5 có cùng giá trị). Sau đó, hàm này gọi `traverse` trên từng nút trong $c[6]$ theo thứ tự đó.

- **`traverse(1)`** : nút 1 có giá trị 10; thêm nó vào S sẽ cho $S = [0, 10]$. Nút 1 không có nút con nào bị chiếm dụng, vì vậy lệnh gọi này kết thúc.
- **`traverse(5)`** : nút 5 có giá trị 20; thêm nó vào S sẽ cho $S = [0, 10, 20]$. Nút 5 có một nút con bị chiếm dụng là nút 2, vì vậy $c[5] = [2]$ và hàm này gọi `traverse(2)`.
 - **`traverse(2)`** : nút 2 có giá trị 30; thêm nó vào S sẽ cho $S = [0, 10, 20, 30]$. Nút 2 không có nút con nào bị chiếm dụng, vì vậy lệnh gọi này kết thúc.
 - Quá trình thực thi quay trở lại `traverse(5)` đã xử lý tất cả các phần tử con trong $c[5]$, do đó lệnh gọi hàm này kết thúc.
- **`traverse(4)`** : nút 4 có giá trị 20; thêm nó vào S sẽ cho $S = [0, 10, 20, 30, 20]$. Nút 4 không có nút con nào bị chiếm dụng, vì vậy lệnh gọi này kết thúc.

Tất cả các lệnh gọi đã kết thúc và không còn nút nào cần xử lý nữa. Cuối cùng, tất cả các quả bóng được lấy ra khỏi máy và `collect` trả về mảng $S = [0, 10, 20, 30, 20]$.

Nhiệm vụ của bạn là xác định số lượng nút N và trả về một mảng các nút cha $R = [R[0], R[1], \dots, R[N-2]]$ mô tả cấu trúc của máy, nhưng với cách đánh chỉ số có thể khác cho các nút không phải là nút lá và không phải là nút gốc. Một cách hình thức, một mảng R được trả về

sẽ coi là đúng nếu có thể đánh chỉ số khác nhau $L[u]$ với $0 \leq L[u] < N$ cho mỗi nút u của máy sao cho:

- $L[u] = u$ với mọi $0 \leq u < M$ và $u = N - 1$, và
- $L[P[u]] = R[L[u]]$ với mọi $0 \leq u < N - 1$.

Không bắt buộc $R[u] > u$. Máy **phải trống** khi bạn trả về lời giải.

Gọi K là số lần thực hiện thao tác `collect`, và B là giá trị lớn nhất của bất kỳ quả bóng nào trong tất cả các lần gọi `insert`. Gọi $C = K + B$. Khi đó, C **không được vượt quá 1000**. Điểm của bạn trong một số subtask phụ thuộc vào giá trị của C .

Chi tiết cài đặt

Bạn cần xây dựng hàm sau.

```
std::vector<int> find_structure(int M)
```

- M : số lượng nút lá trong máy.
- Hàm này được gọi đúng một lần cho mỗi trường hợp test.
- Hàm này phải trả về một mảng $R = [R[0], R[1], \dots, R[N - 2]]$ có độ dài $N - 1$, một mảng cha đúng.

Để tương tác với máy, hàm của bạn có thể gọi hai hàm sau.

Hàm thứ nhất là:

```
bool insert(int U, int X)
```

- U : chỉ số của nút lá mà quả bóng cần được chèn vào. Điều kiện $0 \leq U < M$ phải được thỏa mãn.
- X : giá trị nguyên của quả bóng. Điều kiện $0 \leq X \leq 1000$ phải được thỏa mãn.
- Hàm này trả về `true` nếu quả bóng được đặt thành công, hoặc `false` nếu nút lá U đã bị chiếm dụng.
- Hàm này có thể được gọi tối đa 500 000 lần.

Hàm thứ hai là:

```
std::vector<int> collect()
```

- Thu thập giá trị của tất cả các quả bóng đã được đưa vào, làm trống máy và trả về mảng S đã thu thập.

Nếu tại bất kỳ thời điểm nào trong quá trình thực thi chương trình, giá trị của C vượt quá 1000, giải pháp của bạn sẽ nhận được phản hồi `Output isn't correct: Too many resources used`.

Cấu trúc của máy được cố định trước khi hàm `find_structure` được gọi. Trình chấm điểm là **đơn định** theo nghĩa nếu bạn chạy nó hai lần và cả hai lần chạy đều thực hiện cùng một dãy các thao tác, thì các lệnh gọi đến `collect` sẽ trả về cùng một mảng.

Các ràng buộc

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Các subtask

Subtask	Điểm	Các ràng buộc thêm
1	5	Nút gốc có đúng M nút con.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Không có ràng buộc nào thêm.

Trong các subtask 3 và 4, điểm số của bạn phụ thuộc vào giá trị của C như sau.

Subtask 3 (25 điểm)

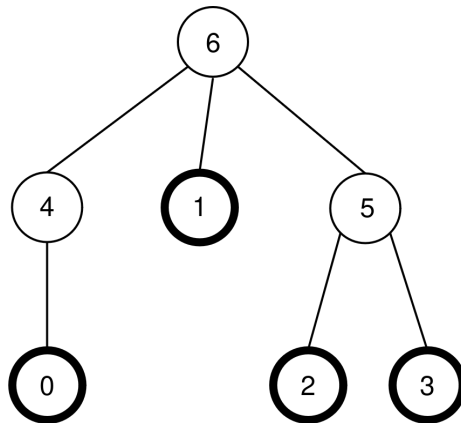
Giới hạn	Điểm
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtask 4 (60 điểm)

Giới hạn	Điểm
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Ví dụ

Xét cùng một máy như trong mô tả nhiệm vụ, với $N = 7$, $M = 4$ và $P = [4, 6, 5, 5, 6, 6]$. Máy đó được minh họa lại trong hình sau:



Trình chấm điểm gọi hàm sau:

```
find_structure(4)
```

Hàm có thể thực hiện dãy cuộc gọi sau:

1. **insert(0, 0)**: nút lá 0 trống, nên một quả bóng có giá trị 0 được đặt tại lá 0. Nút $P[0] = 4$ trống, nên quả bóng di chuyển đến nút 4. Nút $P[4] = 6$ trống, nên quả bóng di chuyển đến nút 6. Vì nút 6 là nút gốc, nên quá trình di chuyển dừng lại. Lệnh gọi trả về `true`.
2. **insert(3, 20)**: nút lá 3 đang trống, nên một quả bóng có giá trị 20 được đặt tại lá 3. Nút $P[3] = 5$ đang trống, nên quả bóng di chuyển đến nút 5. Vì $P[5] = 6$ đang bị chiếm dụng, nên quá trình di chuyển dừng lại. Lệnh gọi trả về `true`.
3. **insert(1, 10)**: nút lá 1 trống, vì vậy một quả bóng có giá trị 10 được đặt tại lá 1. Vì $P[1] = 6$ đã bị chiếm dụng, quả bóng vẫn ở nút 1. Lệnh gọi trả về `true`.
4. **insert(2, 30)**: nút lá 2 đang trống, vì vậy một quả bóng có giá trị 30 được đặt tại lá 2. Vì $P[2] = 5$ đã bị chiếm, quả bóng vẫn ở nút 2. Lệnh gọi trả về `true`.
5. **insert(1, 25)**: nút lá 1 đã bị chiếm dụng, do đó quả bóng không được đặt tại nút lá 1. Lệnh gọi trả về `false`.
6. **insert(0, 20)**: nút lá 0 đang trống, nên một quả bóng có giá trị 20 được đặt tại nút lá 0. Nút $P[0] = 4$ đang trống, nên quả bóng di chuyển đến nút 4. Vì $P[4] = 6$ đã bị chiếm dụng, nên quá trình di chuyển dừng lại. Lệnh gọi trả về `true`.

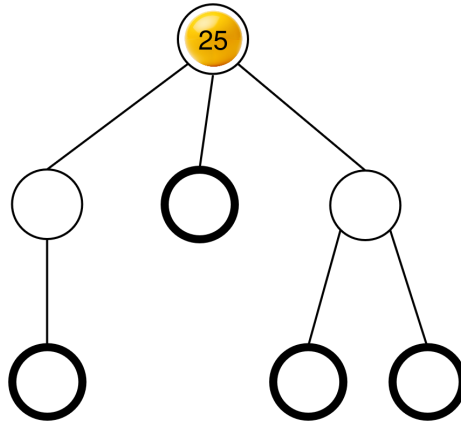
Cấu hình các quả bóng thu được đã được thể hiện trong phần mô tả nhiệm vụ.

Tiếp theo, hàm có thể gọi:

```
collect()
```

Việc thực thi lệnh gọi này đã được giải thích trong phần mô tả nhiệm vụ, vì vậy lệnh gọi này trả về $S = [0, 10, 20, 30, 20]$. Sau đó, máy lại trở về trạng thái trống.

Tiếp theo, hàm có thể gọi `insert(2, 25)`. Nút lá 2 đang trống, vì vậy một quả bóng có giá trị 25 được đặt tại nút lá 2. Quả bóng này sau đó di chuyển đến nút 5, rồi đến nút 6, nơi nó dừng lại. Lệnh gọi trả về `true`. Cấu hình các quả bóng sau khi thực hiện được thể hiện trong hình dưới đây.



Cuối cùng, có thể gọi hàm `collect()`, hàm này trả về giá trị $S = [25]$ và làm trống máy.

Có hai mảng cha hợp lệ mà hàm `find_structure` có thể trả về: $R = P = [4, 6, 5, 5, 6, 6]$ (tương ứng với cách đánh chỉ số $L = [0, 1, 2, 3, 4, 5, 6]$) và $R = [5, 6, 4, 4, 6, 6]$ (tương ứng với cách đánh chỉ số $L = [0, 1, 2, 3, 5, 4, 6]$). Trong ví dụ này, $K = 2$ và $B = 30$, do đó $C = 32$.

Trình chấm mẫu

Định dạng dữ liệu vào:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Định dạng kết quả ra:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Trong đó, Q là kích thước của mảng R được trả về bởi `find_structure`.